best-LORA-Example.py - User Manual

LoRA / QLoRA fine-tuning of causal LLMs with Hugging Face PEFT. Part 11, Section 52. Local Windows CPU machine and Vast.ai GPU. Section 9 covers the vision companion best-VLM-LoRA-Example.py.

1. What the script does

One file, one command. **main()** runs these steps in order:

Step	What happens
1. Data	Load (Hub / local files / save_to_disk folder / streaming) -> detect layout -> clean + de-duplicate -> train/validation split -> prompt or chat template -> tokenize with label masking (loss only on answers) -> optional packing -> DataLoaders
2. Model	Tokenizer (+ pad token) -> base model (fp32 on CPU, bf16/fp16 on GPU, or 4-bit for QLoRA) -> LoraConfig -> get_peft_model (under 1% of weights trainable)
3. Train	AdamW on LoRA weights only, warmup + LR schedule, mixed precision (GPU), gradient accumulation + clipping, validation loss/perplexity, early stopping, best adapter restored
4. Finish	Save adapter -> reload with AutoPeftModelForCausalLM -> generate (LoRA vs base model) -> optional merge into a standalone model

Beginner alternative: **basic-lora-tutorial.py** in the same folder - same idea, no options, top-to-bottom.

2. Run on this machine (Windows, CPU only)

2.1 Requirements (already installed here)

Package	Tested version	Notes
torch	2.12.1 (CPU)	No CUDA on this laptop
transformers	5.12.1	Needs >= 4.56 (uses dtype=, not torch_dtype=)
peft / datasets / accelerate	0.19.1 / 5.0.0 / 1.14.0	
bitsandbytes	not installed	Only for --use_qlora, which needs an NVIDIA GPU

Fresh machine: `pip install "transformers>=4.56" peft datasets accelerate torch`

2.2 Commands

Open a terminal in the Section 52 folder. Outputs go to **./my_lora_adapter** (relative to where you run it).

```
cd "Part 11 - Deep Learning Advanced/Section 52 - LoRA (Low-Rank Adaptation)"

python best-LORA-Example.py --help                # every option with its default

# Quick check (measured: about 4 min here): proves the whole pipeline works
python best-LORA-Example.py --max_train_samples 200 --max_eval_samples 50 --num_epochs 1

# Full default run: GPT-2 on Dolly-15k, 1000 rows, 3 epochs (measured: about 46 min here)
python best-LORA-Example.py

# Save the log as well
python best-LORA-Example.py > train.log
```

Measured default run on this laptop: validation loss 3.07 (base model) -> 2.73 -> 2.71 -> 2.70, best adapter restored and saved, generation clearly follows the trained answer style.

2.3 CPU rules for this laptop

Do	Why
Keep --mixed_precision auto (= off on CPU)	This CPU has AVX2 only; bf16 is emulated and about 100x slower
Run one training job at a time; close heavy apps	16 GB RAM: two jobs at once crashed with exit code 139 (memory pressure). Alone, the same run completed.
Keep models small: gpt2, HuggingFaceTB/SmolLM2-135M-Instruct	GPT-2 + LoRA trains at about 200 tokens/s here
Keep --dataloader_num_workers 0	Fastest on Windows for pre-tokenized data

3. Using your own data

3.1 Where the data comes from (pick one)

Source	Option	Example
Hugging Face Hub	--dataset_name (+ --dataset_config_name)	--dataset_name databricks/databricks-dolly-15k
Local / remote files	--train_file (+ --validation_file)	--train_file data/train.jsonl --validation_file data/val.jsonl
Folder of files	--dataset_name	train.jsonl + validation.jsonl in one folder
save_to_disk() folder	--load_from_disk_path	--load_from_disk_path my_clean_dataset
Huge datasets	--streaming --max_steps N	nothing is downloaded upfront

Precedence if several are set: load_from_disk_path > train_file > dataset_name. Validation: uses a validation/valid/val/dev split if present, otherwise holds out 5% of train (the test split is never used).

3.2 File formats

Extension	Builder	Notes
.csv .tsv	csv	header row = column names
.json .jsonl .ndjson	json	--json_field data when records sit under one key
.parquet	parquet	best for large data
.txt	text	one example per line
.arrow	arrow	Arrow IPC files
.gz .bz2 .xz (.zst)	same as inner file	.zst needs pip install zstandard; globs and URLs also work

3.3 Data layouts (auto-detected from column names)

Layout	Example row	Loss is computed on
text	{"text": "..."}	every token
instruction	{"instruction": "...", "input": "...", "output": "..."} also: prompt/question/query, context, response/completion/answer	the answer only
chat	{"messages": [{"role": "user", "content": "..."}, {"role": "assistant", "content": "..."}]} ShareGPT {"conversations": [{"from": "human", "value": "..."}]} also works	assistant turns only

Different column names: force them, e.g. `--dataset_format instruction --instruction_column question --response_column reply`. The run prints one training example with the trained part wrapped in [[ ]] - check it once.

4. Ready-made commands

On this machine (PowerShell) a long command continues on the next line with a backtick `; on Vast.ai (Linux) the continuation character is \ instead.

```
# Your instruction CSV
python best-LORA-Example.py --train_file data/qa.csv

# Raw text corpus (continued pre-training) with packing
python best-LORA-Example.py --train_file "data/*.parquet" --dataset_format text --packing

# Chat data with a real chat model (CPU-friendly)
python best-LORA-Example.py --dataset_name HuggingFaceH4/no_robots `
       --model_name HuggingFaceTB/SmolLM2-135M-Instruct

# Use every row, longer sequences
python best-LORA-Example.py --max_train_samples 0 --max_eval_samples 0 --max_seq_length 512

# Only test an existing adapter (no training)
python best-LORA-Example.py --do_train false --test_prompt "Explain LoRA in one sentence."

# Also write a standalone merged model (for vLLM / llama.cpp)
python best-LORA-Example.py --save_merged_model
```

```
# All options from a file
python best-LORA-Example.py my_run.json      (or my_run.yaml)
```

Most used options

Option	Default	Meaning
--model_name	gpt2	Any decoder-only LM on the Hub or a local folder
--lora_r / --lora_alpha / --lora_dropout	16 / 32 / 0.1	LoRA rank, scaling (alpha/r), dropout
--lora_target_modules	PEFT default	e.g. q_proj k_proj v_proj o_proj, or all-linear
--max_seq_length	256	Tokens per example (rows whose prompt alone is longer are dropped)
--max_train_samples / --max_eval_samples	1000 / 200	Random subset; 0 = use all
--num_epochs / --max_steps	3 / 0	max_steps > 0 overrides epochs
--per_device_train_batch_size	8	Micro-batch size
--gradient_accumulation_steps	1	Effective batch = batch x steps
--learning_rate	5e-4	LoRA uses higher LR than full fine-tuning
--eval_strategy / --eval_steps	epoch / 50	epoch, steps or no
--early_stopping_patience	2	Evaluations without improvement before stopping; 0 = off
--mixed_precision	auto	bf16/fp16 on CUDA, off on CPU
--use_qlora / --gradient_checkpointing	off / off	4-bit base (GPU only) / less memory
--output_dir	./my_lora_adapter	Where everything is saved

5. What you get

File in --output_dir	Content
adapter_model.safetensors + adapter_config.json	The LoRA adapter (a few MB) and the base model name
tokenizer.json, tokenizer_config.json (+ chat_template.jinja)	Tokenizer exactly as used in training
run_summary.json	Layout, settings, package versions, validation history, best step
checkpoint-best/	Best adapter seen during training (crash-safe copy)
<output_dir>-merged/ (optional)	Standalone full model from --save_merged_model

Reading the log: **[eval] before training** = base model; validation loss should fall after each epoch. If it rises while training loss falls, the model is overfitting and early stopping kicks in.

6. Run on Vast.ai (NVIDIA GPU)

6.1 Rent the instance

Choice	Recommendation
Image / template	A PyTorch (CUDA) template - Linux, Python and torch preinstalled
GPU	RTX 3090 / 4090 (24 GB): GPT-2 up to about 7B with QLoRA. A100/H100 for bigger models. RTX 30xx or newer gets bf16; older cards fall back to fp16 automatically
Disk	30-50 GB (models and datasets are cached on the instance)
Access	SSH (or the Jupyter terminal)

6.2 Set up and run

```
# 1) copy the script up (from your PC; port/host from the Vast.ai SSH button)
scp -P <PORT> best-LORA-Example.py root@<HOST>:/workspace/
#    or: git clone <your repo> on the instance

# 2) on the instance
cd /workspace
pip install -U "transformers>=4.56" peft datasets accelerate bitsandbytes
hf auth login                                  # only for gated models (e.g. Llama)
```

```
# 3) quick GPU check (a minute or two)
python best-LORA-Example.py --max_steps 20 --logging_steps 5

# 4) real run: full Dolly, longer sequences
python best-LORA-Example.py --max_train_samples 0 --max_seq_length 512 > train.log

# 5) QLoRA on a bigger model
python best-LORA-Example.py --model_name Qwen/Qwen2.5-1.5B-Instruct \
       --dataset_name HuggingFaceH4/no_robots --use_qlora --gradient_checkpointing \
       --lora_target_modules all-linear --max_train_samples 0 --max_seq_length 1024

# keep training running after you disconnect
nohup python best-LORA-Example.py --max_train_samples 0 > train.log 2>&1 &
tail -f train.log
```

6.3 Get the results back BEFORE destroying the instance

```
scp -P <PORT> -r root@<HOST>:/workspace/my_lora_adapter .      # a few MB
#  or push it to the Hub from Python: model.push_to_hub("your-user/my-lora")
```

The adapter loads on this laptop too: copy it into the Section 52 folder and run `python best-LORA-Example.py --do_train false` (the base model name is read from adapter_config.json, the prompt layout from run_summary.json).

Status: every CPU path was tested on this machine (about 30 runs + unit checks). The GPU paths (bf16/fp16 mixed precision, QLoRA) follow the library APIs but were not run on a GPU yet - treat the step-3 quick check as the first GPU test.

7. Which models can be fine-tuned

Any decoder-only (causal) text model that `AutoModelForCausalLM` can load - thousands on the Hub, so this is a map of the main families, not a complete list. Pass the Hub id with `--model_name`. **Tested in this session:** gpt2 (default run) and HuggingFaceTB/SmolLM2-135M-Instruct (chat); the others work by the same mechanism but were not run.

Family	Example Hub ids	Sizes	Notes
GPT-2	gpt2, gpt2-medium, distilgpt2	82M-774M	Default; no chat template (the script supplies one)
SmolLM2	HuggingFaceTB/SmolLM2-135M-Instruct, -360M-Instruct, -1.7B-Instruct	135M-1.7B	Best choice for this laptop
Qwen 2.5	Qwen/Qwen2.5-0.5B-Instruct, -1.5B-, -3B-, -7B-, -14B-Instruct	0.5B-72B	Strong, not gated
Qwen 3	Qwen/Qwen3-0.6B, Qwen/Qwen3-1.7B, Qwen/Qwen3-4B, Qwen/Qwen3-8B	0.6B-32B	Template hides earlier 'thinking' text: some multi-turn chats may be dropped (the run reports how many)
Llama 3.x	meta-llama/Llama-3.2-1B-Instruct, -3B-Instruct, meta-llama/Llama-3.1-8B-Instruct	1B-70B	Gated: accept the licence, then hf auth login
Mistral	mistralai/Mistral-7B-Instruct-v0.3	7B	Gated
Gemma	google/gemma-2-2b-it, google/gemma-2-9b-it, google/gemma-3-1b-it	1B-27B	Gated; Gemma 2 template rejects system messages, so those rows are dropped
Phi	microsoft/Phi-3.5-mini-instruct, microsoft/phi-4	3.8B-14B	Not in PEFT's default list: --lora_target_modules qkv_proj o_proj (or all-linear)
TinyLlama	TinyLlama/TinyLlama-1.1B-Chat-v1.0	1.1B	
DeepSeek distills	deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B, -7B	1.5B-70B	
OPT / Pythia / Falcon	facebook/opt-350m, EleutherAI/pythia-410m, tiiuae/falcon-7b	125M-40B	Older base models
Custom-code models	Baichuan, some InternLM / ChatGLM	varies	Need --trust_remote_code

What fits where

Hardware	Practical model size
This laptop (CPU, 16 GB)	Up to about 0.5B: gpt2 or SmolLM2-135M/360M are comfortable; Qwen2.5-0.5B works but is slow
Vast.ai 24 GB (RTX 3090 / 4090), plain LoRA	About 1-8B (add --gradient_checkpointing for 7-8B)
Vast.ai 24 GB, --use_qlora	About 7-14B
Vast.ai 48-80 GB (A6000, A100, H100), QLoRA	30-70B

Will NOT work with this script

Model type	Use instead
Encoder-only: BERT, RoBERTa	lora_finetuning.py in this folder (sequence classification)
Encoder-decoder: T5, BART	AutoModelForSeq2SeqLM + TaskType.SEQ_2_SEQ_LM
Vision-language: Llava, Qwen2-VL, Gemma 3 image models (4B+)	A vision-language training script
Speech models	Section 57 (speech recognition)

Unknown model? Start with `--lora_target_modules all-linear`. If it fails with 'Target modules not found', run `print(model)` to see the real layer names and pass those.

8. Troubleshooting

Message / symptom	Meaning and fix
huggingface_hub 'cache-system uses symlinks' (Windows)	Harmless. Enable Developer Mode or set HF_HUB_DISABLE_SYMLINKS_WARNING=1
'loss_type=None ... Using the default loss: ForCausalLMLoss'	Harmless GPT-2 message; it is the right loss
'dropped N rows with no trainable token left'	The prompt alone is longer than --max_seq_length. Raise it (512 on GPU)
Exit code 139 / process dies on this laptop	Out of memory. Run one job at a time, close other apps, lower batch size
Very slow with --mixed_precision bf16 on CPU	Emulated bf16. Use auto / no on this CPU
'QLoRA needs an NVIDIA GPU' / bitsandbytes error	QLoRA is GPU-only: pip install bitsandbytes on Vast.ai
CUDA out of memory (GPU)	Lower --per_device_train_batch_size and raise --gradient_accumulation_steps; add --gradient_checkpointing; use --use_qlora
'Target modules ... not found'	Wrong layer names for this model: print(model) or use --lora_target_modules all-linear
401 / gated repo	Accept the model licence on the Hub and run hf auth login
'--streaming needs --max_steps'	A stream has no length: add --max_steps N (and --max_eval_samples)
Wrong layout detected	Force it: --dataset_format and the --*_column options

9. Vision-language models: best-VLM-LoRA-Example.py

Companion script in the same folder for image + text models (VLMs). It uses the same training engine as best-LORA-Example.py (answer-only loss, exact gradient accumulation, validation, early stopping, best adapter restored) and puts LoRA on the **language model** of the VLM; the vision encoder stays frozen (--lora_target_modules all-linear adapts it too).

9.1 What is different from the text script

Topic	Vision script
Model loading	AutoModelForImageTextToText + a processor (tokenizer + image processor + chat template). Use an Instruct / chat VLM: models without a chat template are refused.
Image tokens	Every image becomes many tokens inside the prompt (SmolVLM: 64 per 512 px tile, Qwen2.5-VL: about one per 28 x 28 px). --image_longest_edge caps the image size and so the token count.
Images in the data	Kept undecoded while cleaning / splitting (fast), decoded and resized batch by batch in the collator; rows whose image file is missing are dropped.
Truncation	Image tokens are never cut. If prompt + images fill --max_seq_length, the run stops and prints the exact value to use.
Memory	Vocabulary logits are computed only at answer positions (same loss, much less memory).
Outputs	./my_vlm_lora_adapter: adapter + processor + run_summary.json (+ checkpoint-best/).

9.2 Run on this machine (CPU)

```
python best-VLM-LoRA-Example.py --help          # every option with its default

# Default: SmolVLM-256M learns formula image -> LaTeX (unsloth/LaTeX_OCR, 300 images, 38 steps)
python best-VLM-LoRA-Example.py

# Your own data (examples, see 9.3 / 9.4)
python best-VLM-LoRA-Example.py --dataset_name data/my_imagefolder --instruction "Describe the image."
python best-VLM-LoRA-Example.py --train_file data/vqa.jsonl --image_root data/images

# Test a trained adapter on any image (file or URL), no training
python best-VLM-LoRA-Example.py --do_train false --test_image photo.jpg --test_prompt "What is shown?"
```

Measured default run on this laptop: 19 minutes, peak RAM 4.1 GB, validation loss 1.98 -> 1.14. After training the model writes LaTeX in the dataset's style instead of the base model's generic style (not yet accurate after 300 images - that needs the full dataset on a GPU).

CPU rule for this laptop	Why (measured)
Keep the defaults: batch 2 x accumulation 4, --lora_dropout 0, images <= 512 px	Batch 4 + dropout 0.05 peaked at 6.4 GB and crashed; the defaults peak at 4.1 GB
Run ONE training job at a time and close heavy apps	Only about 4.7 GB of 23.5 GB commit memory was free with the usual apps open
Stay with SmolVLM-256M (or 500M) on the CPU	Bigger VLMs need a GPU (Vast.ai)
Do not use --gradient_checkpointing on the CPU unless memory is short	It cut memory 35% but was 2.4x slower

9.3 Data layouts (auto-detected from the column names)

Layout	Example row	Prompt / loss
caption	{"image": ..., "text": "..."} (also caption, description, latex)	prompt = --instruction; loss on the text
vqa	{"image": ..., "question": "...", "answer": "..."} (also query/prompt, label/response; list answers use the first item)	loss on the answer
chat	{"images": [...], "messages": [{"role": "user", "content": [{"type": "image"}, {"type": "text", "text": "..."}]}, {"role": "assistant", ...}]}	loss on assistant turns
chat variants	LLaVA / ShareGPT {"image": "x.jpg", "conversations": [{"from": "human", "value": "<image>\nQ"}, {"from": "gpt", "value": "A"}]}; the_cauldron {"images", "texts": [{"user", "assistant"}]}; several images per row	loss on answers

9.4 Where the images can come from

Source	How to pass it
Hugging Face Hub dataset (embedded images)	--dataset_name HuggingFaceM4/ChartQA (--dataset_config_name for subsets, e.g. the_cauldron ai2d)
Folder of images + metadata.jsonl / metadata.csv	--dataset_name data/my_imagefolder (file_name column + caption or question/answer columns)
JSON / JSONL / CSV with image paths or URLs	--train_file data/vqa.jsonl; relative paths are read from --image_root (default: the folder of the file)
Parquet with embedded images	--train_file data/train.parquet
Folder from save_to_disk()	--load_from_disk_path data/my_dataset
Huge datasets	--streaming --max_steps N --max_eval_samples N (validation = first N rows of the stream)
16-bit / 32-bit / float images (electron / light microscopy, medical, astronomy) and .npy arrays	16-bit or float TIFF / PNG and .npy files work in any source above. They are mapped to 0-255 by --image_scaling (9.6); 8-bit images are used as they are. Vendor files (.dm3 / .dm4 / .emd / .mrc): export to TIFF or .npy first (e.g. with HyperSpy). Multi-page TIFF: first page

9.5 Models

Family	Example Hub ids	Tested here
SmolVLM	HuggingFaceTB/SmolVLM-256M-Instruct, -500M-Instruct, SmolVLM-Instruct (2.2B)	Real 256M model trained on this laptop
Qwen-VL	Qwen/Qwen2-VL-2B-Instruct, Qwen/Qwen2.5-VL-3B-Instruct, -7B-Instruct, Qwen/Qwen3-VL-8B-Instruct	Tiny test versions of Qwen2/2.5/3-VL: all checks
Gemma 3 / Gemma 4	google/gemma-3-4b-it (gated), Gemma 4 vision models	Tiny test versions: all checks
LLaVA / LLaVA-NeXT	llava-hf/llava-1.5-7b-hf, llava-hf/llava-v1.6-mistral-7b-hf	Tiny test versions
Idefics2 / SmolVLM 2	HuggingFaceM4/idefics2-8b, HuggingFaceTB/SmolVLM2-2.2B-Instruct (pip install num2words)	Tiny test versions: all checks
InternVL (-hf format)	OpenGVLab/InternVL3-1B-hf, OpenGVLab/InternVL3-8B-hf	Tiny test version
NOT supported	Llama 3.2 Vision (cross-attention inputs), PaliGemma (no chat template)	Refused with a clear message

Fits: this laptop - SmolVLM-256M / 500M. Vast.ai 24 GB - about 3-4B with plain LoRA, 7-8B with --use_qlora. 48-80 GB (A6000, A100, H100) - larger models with QLoRA.

9.6 All options (python best-VLM-LoRA-Example.py --help)

Most used: --model_name, --dataset_name or --train_file, --instruction, --image_longest_edge, --max_seq_length, --max_train_samples, --per_device_train_batch_size, --use_qlora. Default "-" = not set (auto / off).

Option	Default	Meaning
Model and images		
--model_name	HuggingFaceTB/SmolVLM -256M-Instruct	Instruct VLM from the Hub (needs a chat template), e.g. Qwen/Qwen2.5-VL-7B-Instruct
--model_revision	-	Pin a model commit hash / tag for reproducible runs
--trust_remote_code	false	Allow models that ship their own code (read it first)
--attn_implementation	-	- = library default (sdpa); eager; flash_attention_2 (needs flash-attn)
--model_dtype	auto	Dtype of the frozen base weights: auto = bf16 / fp16 on a GPU, float32 on CPU; or float32, bfloat16, float16
--image_longest_edge	512	Cap on the image's longest side in pixels; fewer image tokens = faster (768-1024 on a GPU)
--image_scaling	percentile	Images with more than 8 bits per value (16-bit / float TIFF or PNG, .npy) -> 0-255: percentile = contrast stretch per image; fixed = one raw range for all. 8-bit images are unchanged

Option	Default	Meaning
--image_percentiles	0.5 99.5	percentile scaling: LOW HIGH percentiles that become black / white (ignores hot pixels; 0 100 = min-max)
--image_value_range	-	fixed scaling: raw LOW HIGH values, e.g. 0 4095 (12-bit detector) or 0 65535; keeps brightness comparable between images
Data source (if several are set: load_from_disk_path, then train_file, then dataset_name)		
--dataset_name	unsloth/LaTeX_OCR	Hub dataset id, or a local folder (imagefolder: images + metadata.jsonl / .csv)
--dataset_config_name	-	Hub subset name, e.g. ai2d for HuggingFaceM4/the_cauldron
--dataset_revision	-	Pin a dataset commit / tag / branch
--train_file	-	Path, glob or URL: .json .jsonl .csv .tsv .parquet .arrow (may be compressed)
--validation_file	-	Optional validation file, same format as --train_file
--load_from_disk_path	-	Folder written by Dataset / DatasetDict.save_to_disk()
--image_root	-	Base folder for relative image paths; - = folder of --train_file / the local dataset, else the current folder
--json_field	-	JSON only: key that holds the list of records, e.g. data
--train_split	train	Name of the training split
--eval_split	-	Validation split; - = validation / valid / val / dev if present, else held out from train
--streaming	false	Read rows lazily (huge datasets); needs --max_steps and --max_eval_samples
--cache_dir	-	Folder for downloads and processed caches (- = ~/.cache/huggingface)
Data layout (9.3)		
--dataset_format	auto	auto = detect from the column names; or caption, vqa, chat
--images_column	-	Column with the image(s), path(s) or URL(s); - = auto-detect
--caption_column	-	Caption / target text column (caption layout); - = auto-detect
--question_column	-	Question column (vqa layout); - = auto-detect
--answer_column	-	Answer column (vqa layout), lists use the first item; - = auto-detect
--messages_column	-	Conversation column (chat layout); - = auto-detect
--instruction	Write the LaTeX representation for this image.	User prompt for the caption layout - change it to match your task
--system_prompt	-	Optional system message added to every example
Preprocessing		
--max_seq_length	1024	Max tokens per example INCLUDING image tokens; only the answer side is ever cut
--deduplicate	true	Drop exact duplicates (same text + same image) before splitting
--validation_split_percentage	5	Percent of train held out when the source has no validation split
--max_train_samples	300	Random subset for quick runs; 0 = every example
--max_eval_samples	40	Validation subset; 0 = every example (streaming: rows taken from the stream)
--preprocessing_num_workers	-	Processes for cleaning / hashing big datasets
--overwrite_cache	false	Ignore the cached processed datasets and recompute them
--shuffle_buffer_size	1000	Streaming only: rows held in the shuffle buffer
LoRA / QLoRA		
--lora_r	16	LoRA rank r (capacity of the adapter)
--lora_alpha	32	LoRA alpha; the update is scaled by alpha / r
--lora_dropout	0	Dropout on the LoRA branch; 0.05-0.1 regularizes but costs memory and ~25% speed

Option	Default	Meaning
--lora_target_modules	-	- = every linear layer of the language model; all-linear = also the vision encoder; or layer names
--use_qlora	false	Load the frozen base model in 4-bit (NVIDIA GPU + bitsandbytes only)
--gradient_checkpointing	false	Much less memory, roughly 30% slower
Training		
--output_dir	./my_vlm_lora_adapter	Where the adapter, processor and run_summary.json are saved
--do_train	true	false = skip training; only load and test the adapter in --output_dir (needs --test_image)
--num_epochs	1	Passes over the training data
--max_steps	0	> 0 overrides --num_epochs (required with --streaming)
--per_device_train_batch_size	2	Examples per training micro-batch; 2 fits a 16 GB laptop, try 4 on a GPU
--per_device_eval_batch_size	4	Examples per validation batch
--gradient_accumulation_steps	4	Micro-batches per optimizer step; effective batch = batch size x this
--learning_rate	0.0002	Peak learning rate
--weight_decay	0.01	AdamW weight decay
--lr_scheduler_type	linear	linear, cosine, constant_with_warmup, ...
--warmup_ratio	0.1	Share of all optimizer steps used to warm the learning rate up from 0
--max_grad_norm	1	Gradient clipping; 0 = off
--mixed_precision	auto	auto = bf16 (fp16 on older GPUs) on a GPU, off on CPU; or no, bf16, fp16
--eval_strategy	steps	When the validation loss is computed: steps, epoch, or no (then nothing is held out)
--eval_steps	10	Evaluate every N optimizer steps (with --eval_strategy steps)
--early_stopping_patience	3	Stop after N evaluations in a row without improvement; 0 = off
--early_stopping_threshold	0	Minimum validation-loss decrease that counts as an improvement
--logging_steps	5	Print the training loss every N optimizer steps
--dataloader_num_workers	-	Image-processing workers; - = 0 on Windows, 4 on Linux / macOS (Vast.ai)
--seed	42	Seed for splits, shuffling, LoRA init and sampling
Testing after training / inference		
--num_test_samples	2	Validation images answered after training, shown next to the reference answer
--test_image	-	Also answer for this image file or URL
--test_prompt	-	Prompt for --test_image (- = --instruction)
--max_new_tokens	128	Tokens to generate
--do_sample	false	false = greedy decoding (best for OCR / extraction); true = sampling
--temperature	0.7	Sampling temperature (only with --do_sample true)
--top_p	0.9	Nucleus sampling threshold (only with --do_sample true)
--repetition_penalty	1.05	> 1.0 discourages repeated tokens
--compare_with_base	true	Also answer with the adapter switched off (= the base model)
--save_merged_model	false	Also save base + adapter merged into one model in <output_dir>-merged

On / off options: --use_qlora (or --use_qlora true) switches one on; --do_train false switches one off. Lists take values separated by spaces: --image_percentiles 1 99. All options can also come from one file: python best-VLM-LoRA-Example.py my_config.json (or .yaml).

9.7 Vast.ai (NVIDIA GPU)

```
pip install -U "transformers>=4.56" peft datasets accelerate pillow bitsandbytes num2words

# quick GPU check
python best-VLM-LoRA-Example.py --model_name Qwen/Qwen2.5-VL-3B-Instruct --max_steps 20 --logging_steps 5

# QLoRA, 7B model, full LaTeX_OCR dataset, keep running after disconnect
nohup python best-VLM-LoRA-Example.py --model_name Qwen/Qwen2.5-VL-7B-Instruct --use_qlora \
      --gradient_checkpointing --image_longest_edge 768 --max_seq_length 2048 --max_train_samples 0 \
      --per_device_train_batch_size 4 --gradient_accumulation_steps 2 > vlm.log 2>&1 &

# charts question answering (ChartQA has its own validation split)
python best-VLM-LoRA-Example.py --model_name Qwen/Qwen2.5-VL-3B-Instruct --dataset_name HuggingFaceM4/ChartQA \
      --max_train_samples 0

# copy the adapter back before destroying the instance
scp -P <PORT> -r root@<HOST>:/workspace/my_vlm_lora_adapter .
```

On Linux the DataLoader uses 4 worker processes by default, so image decoding runs in parallel with the GPU.

9.8 Vision troubleshooting

Message / symptom	Meaning and fix
'... has no chat template'	Base / non-chat model: use an Instruct VLM
'... rows have NO answer tokens left ... Use --max_seq_length N'	Prompt + image tokens fill the limit: use the printed N or lower --image_longest_edge
Exit code 139 / 'access violation' on this laptop	Out of memory: one job only, close apps, --per_device_train_batch_size 1 --gradient_accumulation_steps 8
CUDA out of memory	Lower batch size (raise accumulation), --image_longest_edge, --gradient_checkpointing, --use_qlora
Microscope / medical images too dark, too bright or noisy	16-bit / float images are contrast-stretched (--image_percentiles 0.5 99.5). Try 1 99, or --image_scaling fixed --image_value_range with the detector's range
'[load] WARNING: this adapter was trained with --image_...'	Inference with other image settings than training: pass the same values
'Package num2words is required'	SmolVLM 2 processor: pip install num2words
'per-token inputs [...] that this script does not handle'	Llama 3.2 Vision: not supported - use another family
'[ERROR] `loss` is part of ... signature, but not documented'	Harmless transformers 5.12 message for Qwen-VL
'pad_token_id must be None or an integer within the vocabulary'	Harmless SmolVLM config message
First run slow before training starts	The cleaning pass renders every row once (68k rows for LaTeX_OCR); it is cached for later runs

Status: every CPU path was run on this machine (default run, ~20 integration runs over all data formats and Hub streaming, 70 unit checks, 58 image bit-depth checks, short runs on 16-bit / float TIFF and .npy micrographs). GPU paths (bf16 / fp16, QLoRA) follow the library APIs but were not run on a GPU yet - treat the quick GPU check as the first GPU test.